Guia do código
Gateway DCIM

Monitoramento de data center com multicast, UDP, gRPC e TCP, tudo em Protocol Buffers

TI0151 · Sistemas Distribuídos (2026.2) · UFC
Material de estudo para a arguição
Repositório: github.com/JustinoCarneiro/gateway-dcim-sd

Como usar este guia. O professor vai perguntar o **porquê de cada decisão** (Protobuf, linguagem, comunicação entre os elementos). Por isso, além de explicar o que cada arquivo e cada função fazem, o guia traz, em cada ponto, a justificativa da escolha e, no final, uma lista de perguntas prováveis com respostas curtas, uma seção sobre os dispositivos (as fontes de dados) e um roteiro para deixar tudo rodando antes de a arguição começar.

Índice

1. O sistema em uma página
2. Organização das pastas e dos arquivos
3. Como compilar e executar
4. O contrato: `datacenter.proto`
5. Classe por classe, função por função
6. Fluxos ponta a ponta
7. Concorrência: quem roda em qual thread
8. Decisões e justificativas (perguntas prováveis)
9. Os dispositivos (fontes de dados)
10. Limitações que vale admitir
11. Mapa: pergunta → onde está no código
12. Antes da arguição: deixar tudo rodando

1. O sistema em uma página

Domínio: monitoramento de data center (DCIM, *Data Center Infrastructure Management*). Um **gateway** central descobre **fontes de dados**, recebe as leituras delas, comanda a fonte controlável e responde a um **cliente** de linha de comando.

Processo	Classe	Papel	Quantos na demonstração
Gateway	`Gateway`	Centro do sistema: descoberta, recepção UDP, servidor TCP, cliente gRPC, histórico	1
Ar-condicionado	`ArCondicionado`	Fonte **controlável** (servidor gRPC) que também envia leituras	1 (`ar-01`)
Sensor de temperatura	`SensorTemperatura`	Fonte contínua por rack, com evento imediato ao cruzar 27 °C	2 (racks A e B)
PDU	`Pdu`	Fonte contínua: consumo do rack em watts	2 (racks A e B)
Cliente	`Cliente`	Terminal do operador: listar, comandar, consultar	1
Painel (extra)	`Painel`	Ponte HTTP entre a página da demonstração e os processos reais. Não faz parte do sistema	opcional

Os cinco canais de comunicação

Canal	Transporte e endereço	Mensagem (.proto)	Por que esse transporte
Descoberta (pedido)	UDP **multicast** 239.255.42.99:5007	`DescobertaSolicitacao`	O gateway não conhece os IPs das fontes: pergunta a um grupo e quem estiver escutando responde
Descoberta (resposta)	UDP **unicast** ao remetente do pedido	`DescobertaResposta`	Só o gateway precisa da resposta; não faz sentido inundar o grupo
Leituras	UDP unicast fonte → gateway :5008	`Leitura`	Frequente, pequena, perda ocasional é tolerável; sem conexão por fonte

Comandos	**gRPC** gateway → ar-condicionado :50051	`ArCondicionado.Configurar` / `ObterEstado`	Comando exige resposta confiável, contrato tipado e códigos de erro
Cliente	**TCP** cliente → gateway :5009	`PedidoCliente` / `RespostaCliente`	Conversa de pedido e resposta, que precisa ser confiável e ordenada

Regra do enunciado cumprida: todas as mensagens de rede, inclusive a descoberta e o cliente TCP, são Protocol Buffers. O único conteúdo que não é Protobuf é o arquivo CSV local do gateway, que não trafega pela rede.

2. Organização das pastas e dos arquivos

```
gateway-java/
├── pom.xml                       build Maven: dependências e geração do código Protobuf/gRPC
├── rodar.sh                      atalho para subir qualquer classe: ./rodar.sh Gateway
├── README.md                     instruções de instalação, execução e roteiro da demonstração
├── .gitignore                    ignora target/, data/ e arquivos de IDE
│
├── src/main/proto/
│   └── datacenter.proto          O CONTRATO: todas as mensagens e o serviço gRPC (um único
arquivo)
│
├── src/main/java/dc/             todo o código-fonte (um único pacote, "dc")
│   ├── Gateway.java              processo central
│   ├── Cadastro.java             estado do gateway em memória (fontes, leituras, consultas)
│   ├── HistoricoCsv.java         persistência das leituras em CSV
│   ├── Cliente.java              CLI do operador
│   ├── ArCondicionado.java       fonte controlável (servidor gRPC)
│   ├── SensorTemperatura.java    fonte contínua de temperatura
│   ├── Pdu.java                  fonte contínua de potência
│   ├── DescobertaFonte.java      lado da fonte na descoberta (compartilhado pelas 3 fontes)
│   ├── Quadro.java               enquadramento de mensagens no TCP (4 bytes de tamanho)
│   ├── Rede.java                 escolha da interface de rede para o multicast
│   ├── Config.java               portas, endereços e parâmetros em um só lugar
│   └── Painel.java               servidor HTTP da demonstração (extra)
│
├── docs/
│   ├── slides.html / slides.pdf     os 8 slides da apresentação (o PDF é gerado do HTML)
│   ├── gerar_pdf.sh                 gera slides.pdf com o Chrome em modo headless
│   ├── demo-online.html             demonstração no navegador: simulação ou Java real (via
Painel)
│   ├── guia-do-codigo.html / .pdf   este guia
│   └── lm-fontes.css, ufc_logo.png  fontes e logotipo dos slides
│
├── data/leituras.csv             GERADO em execução: histórico das leituras (fora do Git)
└── target/                       GERADO pelo Maven: classes compiladas e código Protobuf/gRPC
(fora do Git)
```

Pontos que o professor pode perguntar. (1) **As classes `Leitura`, `DescobertaResposta` etc. não estão em `src/`**: o `mvn compile` as gera a partir do `.proto` em `target/generated-sources` (pacote `dc.proto`). Isso garante que todos os processos usem exatamente o mesmo contrato. (2) **Um pacote só**: o sistema é pequeno e cada classe é um processo ou um apoio direto, então separar em subpacotes só traria burocracia.

Quem é processo e quem é apoio

Tem `main` (vira processo)	Só apoio (usada por outras classes)
`Gateway`, `ArCondicionado`, `SensorTemperatura`, `Pdu`, `Cliente`, `Painel`	`Cadastro`, `HistoricoCsv`, `DescobertaFonte`, `Quadro`, `Rede`, `Config`

Origem do CSV: a ideia de gravar as leituras em arquivo e consultar o histórico veio do projeto de um colega; aqui foi reimplementada sobre o modelo de dados deste projeto (`Leitura` tipada, hora de chegada do gateway).

3. Como compilar e executar

Compilação

- `mvn -q compile` faz duas coisas: o `protobuf-maven-plugin` baixa o `protoc` (compilador do Protobuf) e o plugin do gRPC, gera o código Java em `target/generated-sources` e depois o `maven-compiler-plugin` compila tudo (Java 21).
- O `os-maven-plugin` descobre o sistema operacional para baixar o `protoc` do tipo certo (Linux, Windows, Mac).
- Versões: gRPC 1.68.0 e protobuf-java 3.25.5. Dependência extra `annotations-api` (escopo *provided*): o código gRPC gerado usa `@Generated`, que saiu do JDK.

Execução: `rodar.sh`

Script de 4 linhas: entra na pasta do projeto, compila se `target/classes/dc` não existir, gera o classpath uma vez em `target/cp.txt` (`mvn dependency:build-classpath`) e executa `java $JAVA_OPTS -cp "target/classes:$(cat target/cp.txt)" dc.<Classe> args…`. Assim cada processo sobe em segundos, sem passar pelo Maven.

Comando	O que faz
`./rodar.sh ArCondicionado ar-01`	gRPC na porta 50051; o argumento é o id da fonte
`./rodar.sh SensorTemperatura temp-rack-A A`	id da fonte e rack; digitar `q` esquenta o rack, `n` normaliza
`./rodar.sh Pdu pdu-rack-A A`	id e rack; `q` = pico de consumo, `n` = normal
`./rodar.sh Gateway`	sobe descoberta, UDP :5008 e TCP :5009
`./rodar.sh Cliente [ip-do-gateway]`	CLI; sem argumento conecta em 127.0.0.1
`./rodar.sh Painel`	servidor HTTP em `http://localhost:8099/` para a demonstração no navegador

Opções `-D` (via `JAVA_OPTS`)

Opção	Efeito
`-Ddc.interface=wlo1`	força a interface de rede do multicast (use o **mesmo** valor em todos os processos)
`-Ddc.descoberta.s=10`	repete o pedido de descoberta a cada 10 s em vez de 30 s (só o gateway)
`-Ddc.csv=/caminho/leituras.csv`	muda o arquivo do histórico (padrão: `data/leituras.csv`)

4. O contrato: `datacenter.proto`

É o arquivo central: define **todas** as mensagens trocadas e o serviço gRPC. Sintaxe `proto3`, pacote `datacenter`, `java_multiple_files = true` (uma classe Java por mensagem) e `java_package = "dc.proto"`. Os números dos campos (`= 1`, `= 2`…) são o que vai no fio: o nome do campo não trafega, por isso a mensagem é compacta e, uma vez publicado, um número nunca deve ser reaproveitado.

Enums

Enum	Valores	Para quê
`TipoFonte`	AR_CONDICIONADO, SENSOR_TEMPERATURA, PDU	tipo da fonte (o `0` é "desconhecido", exigência do proto3)
`Grandeza`	TEMPERATURA_C, POTENCIA_W	o que a leitura mede; unidades fixas (°C e W) evitam conversão
`EstadoFonte`	ATIVA, INATIVA	estado calculado pelo gateway
`TipoConsulta`	TEMPERATURA_MEDIA, RACK_MAIS_QUENTE, PICO_ENERGIA	as 3 consultas agregadas

> **Por que enum e não texto?** Um enum ocupa 1 byte no fio, o compilador recusa valores errados e não há risco de "Temperatura" ≠ "temperatura". Texto livre exigiria validação manual em cada ponta.

Mensagens de descoberta (multicast)

Mensagem	Campos	Observações
`DescobertaSolicitacao`	`gateway_ip`, `gateway_porta_udp`, `nonce`	O pedido já informa **para onde enviar as leituras**: a fonte não precisa de configuração
`DescobertaResposta`	`fonte_id`, `tipo`, `ip`, `porta`, `controlavel`, `rack`, `intervalo_ms`, `optional estado_inicial`, `nonce`	Tipo, IP, porta e estado inicial, como o enunciado pede. `porta` é a do gRPC (0 se não controlável). O `nonce` é cópia do pedido
`EstadoAr`	`ligado`, `setpoint_c`, `ventilador_pct`, `intervalo_ms`	Estado do ar-condicionado; usado na descoberta, nas leituras e na resposta do gRPC

Leitura (UDP)

Campo	Significado
`fonte_id`, `tipo`, `rack`	quem mediu
`grandeza`, `valor`	o que e quanto (°C ou W)
`timestamp_ms`	hora da medição, no relógio da **fonte** (informativo; o gateway usa a hora de chegada)
`evento`	`true` = envio imediato por cruzamento de limiar

`intervalo_ms`	período de envio vigente; base do critério de inatividade
`seq`	contador por fonte; permite ao gateway perceber perda, duplicação e reordenação
`optional estado_ar`	só o ar-condicionado preenche

Serviço gRPC

```
service ArCondicionado {
  rpc Configurar(ConfigurarRequest) returns (EstadoAr);
  rpc ObterEstado(Vazio)            returns (EstadoAr);
}
message ConfigurarRequest {            // todos optional: ausente = não alterar
  optional bool   ligado         = 1;
  optional double setpoint_c     = 2;  // 10..35
  optional uint32 ventilador_pct = 3;  // 0..100
  optional uint32 intervalo_ms   = 4;  // 500..60000
}
```

Por que `optional`? No proto3 sem `optional`, `0` e "campo ausente" são indistinguíveis: não dá para diferenciar "ventilador a 0 %" de "não mexa no ventilador". Com `optional` há `hasX()`, o que permite **atualização parcial**. **Por que valores absolutos (e não "aumente 2 graus")?** Repetir o comando depois de um timeout dá o mesmo resultado: o comando é **idempotente**.

Cliente ↔ gateway (TCP)

Mensagem	Conteúdo
`PedidoCliente`	`oneof pedido`: `listar` (1), `comando` (2), `consulta` (3), `historico` (4)
`RespostaCliente`	`ok`, `erro` e `oneof resposta`: `listar` (3), `comando` (4), `consulta` (5), `historico` (6)
`FonteInfo`	visão de uma fonte: id, tipo, ip, porta, controlável, rack, **estado** (ATIVA/INATIVA), `ultima_leitura_ms`, `optional estado_ar`
`ConsultaRequest` / `ConsultaResponse`	tipo + `janela_s` (0 = 300 s) → `tem_dados`, `valor`, `referencia`, `amostras`
`HistoricoRequest` / `HistoricoResponse`	`fonte_id` ("" = todas) e `limite` (0 = 20) → lista de `LeituraRegistrada` (hora de chegada + `Leitura`)

Por que `oneof`? O TCP carrega um único tipo de mensagem (`PedidoCliente`); o `oneof` diz qual dos pedidos veio, sem precisar de um campo "tipo" manual, e o Java gera `getPedidoCase()` para o `switch`. **Por que `ok` / `erro` na resposta?** Um erro de negócio (fonte inexistente, valor recusado) não deve derrubar a conexão: volta como `ok = false` com a mensagem. **Por que `tem_dados`?** Média sem leituras na janela não é 0,0: é "sem dados".

5. Classe por classe, função por função

5.1 `Config`: constantes compartilhadas

Classe `final` sem instância, só constantes: um ponto único para mudar portas e parâmetros.

Constante	Valor	Observação
`MULTICAST_GRUPO` / `MULTICAST_PORTA`	239.255.42.99 / 5007	faixa 239.0.0.0/8 é a de multicast de escopo administrativo (local)
`GATEWAY_UDP_PORTA` / `GATEWAY_TCP_PORTA`	5008 / 5009	leituras / cliente
`AR_GRPC_PORTA_PADRAO`	50051	porta convencional do gRPC
`TOLERANCIA_INATIVIDADE`	3	intervalos sem leitura até marcar INATIVA
`TAMANHO_MAX_QUADRO`	1 MiB	limite de um quadro TCP
`JANELA_PADRAO_S`	300	janela das consultas quando o cliente não informa
`RETENCAO_MS`	1 hora	quanto histórico fica na memória e é recarregado do CSV
`JANELA_DESCOBERTA_MS` / `INTERVALO_DESCOBERTA_S`	3000 / 30	quanto o gateway espera respostas / de quanto em quanto repete o pedido
`CSV_CAMINHO`, `HISTORICO_PADRAO`, `HISTORICO_MAXIMO`	data/leituras.csv, 20, 500	persistência e limites do `historico`

5.2 `Rede`: interface de rede

Método	O que faz
`escolherInterface()`	Se existir `-Ddc.interface`, usa essa interface. Senão, devolve a primeira que esteja ativa, aceite multicast, não seja loopback e tenha IPv4. Sem nenhuma, lança erro com a instrução de usar `-Ddc.interface`
`ipv4(ni)`	Primeiro endereço IPv4 da interface, ou `null`

> **Por quê?** Um computador tem várias interfaces (Wi-Fi, Docker, loopback) e o multicast sai por **uma** só; se gateway e fontes escolherem interfaces diferentes, não se enxergam. Por isso a escolha é explícita e igual em todos os processos. Os processos também forçam `java.net.preferIPv4Stack=true` (bloco `static`) para que o Java use IPv4 de forma consistente.

5.3 `Quadro`: mensagens no TCP

Método	O que faz
`enviar(out, msg)`	Serializa a mensagem, monta `[4 bytes big-endian com o tamanho]` `[corpo]` e envia num **único** `write` + `flush`
`receber(in, parser)`	Lê 4 bytes (`readInt`), confere o limite (0 a 1 MiB), lê exatamente o corpo (`readFully`) e decodifica com o parser da mensagem esperada
`QuadroInvalidoException`	Tamanho fora do limite: o fluxo perdeu o alinhamento e só resta **fechar a conexão**

> **Por que 4 bytes de tamanho?** TCP é um **fluxo de bytes**: uma mensagem pode chegar em pedaços ou duas podem chegar coladas; e o Protobuf não marca onde termina. O prefixo de tamanho resolve as duas coisas. `readFully` porque um `read` simples pode devolver só parte do quadro. **Limite de 1 MiB** para que um cliente defeituoso (ou malicioso) não faça o gateway alocar 2 GiB. **Conteúdo inválido com tamanho válido** não derruba a conexão: o quadro já foi consumido, o fluxo segue alinhado.

5.4 `DescobertaFonte`: a fonte na descoberta

Implementa `Runnable`; roda numa thread própria em cada fonte.

Parte	O que faz
Construtor	Recebe um `Supplier` que produz a resposta (é *Supplier* porque o estado do ar-condicionado muda), a interface de rede e um *callback* chamado a cada pedido válido
`run()`	Cria um `MulticastSocket` com `setReuseAddress(true)`, liga na porta 5007 e entra no grupo 239.255.42.99 pela interface escolhida. Em laço: recebe um datagrama, tenta decodificar como `DescobertaSolicitacao` (inválido ou sem IP/porta de gateway: ignora), chama o callback e responde por **unicast** ao endereço de origem do datagrama, com o `nonce` copiado do pedido

> **Por que `setReuseAddress`?** Na demonstração as 5 fontes rodam na mesma máquina e todas precisam escutar a mesma porta do grupo. **Para que serve o callback?** A fonte **aprende** o IP e a porta UDP do gateway a partir do pedido: não há configuração manual do destino das leituras.

5.5 `ArCondicionado`: a fonte controlável

Threads: (1) `descoberta` (`DescobertaFonte`); (2) pool do gRPC, que atende `Configurar` / `ObterEstado`; (3) a principal, que simula a temperatura e envia leituras. O estado (ligado, setpoint, ventilador, intervalo, temperatura) é compartilhado e todo acesso é `synchronized`.

Método	O que faz
`main`	Lê o id (padrão `ar-01`) e a porta gRPC (padrão 50051) e chama `executar`
`executar(id, porta)`	Escolhe a interface, sobe o servidor gRPC, cria a thread de descoberta (a resposta anuncia tipo, IP, porta gRPC, `controlavel = true`, intervalo e estado atual) e entra no laço de leituras
`estado()`	Monta o `EstadoAr` atual
`validar(req)`	Devolve a mensagem de erro se algum campo **presente** estiver fora da faixa (setpoint 10–35 °C, ventilador 0–100 %, intervalo 500–60000 ms); senão `null`
`aplicar(req)`	Aplica só os campos presentes (`hasX()`) e devolve o novo estado
`avancar(dt)`	Modelo físico simples: a temperatura tende ao setpoint se ligado, ou a 32 °C se desligado; o ventilador acelera a convergência; mais um ruído pequeno
`Servico.configurar`	Valida (erro vira `Status.INVALID_ARGUMENT`), aplica, registra no log e responde com o novo `EstadoAr`
`Servico.obterEstado`	Devolve o `EstadoAr` atual (definido no contrato; o gateway não o usa, veja a seção 10)

`enviarLeituras(id)`	Laço a cada 100 ms: avança a temperatura; se já passou o intervalo e o gateway é conhecido, monta uma `Leitura` (com `seq` crescente e `estado_ar` anexado) e a envia por `DatagramSocket`

> **Por que o ar continua enviando leituras quando está desligado?** Para o gateway distinguir **"desligado por comando"** (continua enviando, com `ligado=false`) de **"caiu"** (parou de enviar). Se parasse de enviar ao ser desligado, ficaria INATIVA e seria impossível diferenciar os dois casos. **Por que validar na fonte e não no gateway?** A fonte é quem conhece seus limites físicos; o gateway não duplica a regra e o erro volta pelo próprio gRPC.

5.6 `SensorTemperatura`: fonte contínua com evento

Threads: descoberta, teclado (só para a demonstração) e a principal. Argumentos: id e rack.

Método	O que faz
`executar(id, rack)`	Monta a resposta de descoberta (tipo sensor, `porta = 0`, `controlavel = false`, rack, intervalo 2000 ms), cria as threads de descoberta e de teclado e chama `enviarLeituras`
`lerTeclado()`	Lê o stdin: `q` muda a base de temperatura para 29 °C (rack esquentando) e `n` volta para 22 °C
`enviarLeituras(id, rack)`	Amostra a cada 200 ms: valor = base ± 1 °C. Envia (a) **periodicamente**, a cada 2 s, e (b) **imediatamente** quando o estado "acima de 27 °C" muda, nos dois sentidos, marcando `evento = true`

> **Por que 27 °C?** É o limite superior da faixa de temperatura recomendada para equipamentos de TI segundo a ASHRAE (TC 9.9). **Por que evento nos dois sentidos?** Quem opera precisa saber quando o problema **começou** e quando **acabou**. **Por que amostrar a cada 200 ms se envia a cada 2 s?** Para detectar o cruzamento do limiar rápido, sem esperar o próximo envio periódico.

5.7 `Pdu`: fonte contínua de potência

Mesma estrutura do sensor, sem eventos: a cada 2 s envia o consumo (base 3500 W ± 150 W; `q` = 6500 W de pico, `n` = normal). Grandeza `POTENCIA_W`, tipo `PDU`. Serve ao tipo "contínua, sem evento" e à consulta `pico`.

5.8 `Gateway`: o processo central

Threads: `udp-leituras`; `tcp-clientes` (aceita conexões e cria **uma thread por cliente**); um agendador que roda `descobrir()` a cada 30 s; e a thread `main`, que só prepara tudo e termina.

Método	O que faz
`main`	Cria o `Gateway`; **recarrega do CSV** a última hora de leituras para o `Cadastro`; sobe as threads UDP e TCP **antes** da descoberta (para não perder leituras de fontes que respondam rápido); agenda a descoberta com `scheduleWithFixedDelay`, começando em 0 s
`descobrir()`	Escolhe a interface; sorteia um `nonce`; monta a `DescobertaSolicitacao` (IP do gateway, porta UDP 5008, nonce); abre um `MulticastSocket` de porta efêmera e envia ao grupo; durante 3 s recebe respostas, descarta as de nonce diferente e cadastra as novas (`[descoberta] nova fonte`)

`receberLeituras()`	Escuta UDP :5008. Para cada datagrama: decodifica `Leitura`; descarta se não tem `fonte_id` ou grandeza; chama `cadastro.registrarLeitura`; **grava no CSV**; mostra no log e avisa anomalias de `seq`
`servirClientes()`	Abre o `ServerSocket` :5009 e, a cada conexão, cria uma thread que executa `atender`
`atender(socket)`	Laço: lê um `PedidoCliente` com `Quadro.receber`, responde com `Quadro.enviar`. Conteúdo inválido → resposta de erro e a conexão segue; fim da conexão → log; quadro fora do limite → fecha
`processar(pedido)`	`switch` pelo tipo do `oneof`: `LISTAR` → `cadastro.listar()`; `COMANDO` → `comandar`; `CONSULTA` → `cadastro.consultar`; `HISTORICO` → `historico`; vazio → erro
`comandar(cmd)`	Confere se a fonte existe e é controlável; abre um canal gRPC até o IP e a porta **descobertos**; chama `configurar` com prazo de 3 s; guarda o estado devolvido. `INVALID_ARGUMENT` vira "fonte recusou o comando"; qualquer outro erro vira "fonte não respondeu" (ex.: `UNAVAILABLE`). Fecha o canal no `finally`
`historico(req)`	Lê as últimas leituras do CSV (`csv.ultimos`), limitadas a 500, e as devolve em `HistoricoResponse`
`erro(msg)`	Constrói uma `RespostaCliente` com `ok = false`

> **Por que o gateway usa a porta efêmera na descoberta, e não a 5008?** O datagrama Protobuf não carrega o tipo da mensagem; se `DescobertaResposta` e `Leitura` chegassem no mesmo socket, seria impossível distinguir uma da outra. Portas separadas resolvem. **Por que repetir a descoberta?** UDP perde pacotes e fontes podem subir depois do gateway. **Por que o prazo de 3 s no gRPC?** Sem prazo, uma fonte travada deixaria o cliente esperando para sempre. **Por que um canal por comando?** Simplicidade; com muitos comandos valeria reutilizar o canal (anotado no código).

5.9 `Cadastro`: o estado em memória

Guarda as fontes descobertas e o histórico recente. **Todos os métodos públicos são `synchronized`**, porque três tipos de thread mexem aqui ao mesmo tempo (descoberta, UDP, clientes TCP). O histórico é uma `Deque` de `Registro(leitura, recebidoEm)` em ordem de chegada.

Método	O que faz
`registrarFonte(r)`	Guarda/atualiza a fonte; na primeira vez, marca o instante da descoberta. Devolve `true` se for nova
`registrarLeitura(l, recebidoEm)`	Confere o `seq` (duplicada, fora de ordem, perdida; `seq 0` = fonte reiniciou); adiciona ao histórico; **descarta o que passou de 1 h**; atualiza última leitura, intervalo e estado do ar. Devolve a descrição da anomalia, ou `null`
`restaurar(antigas)`	Repõe no histórico as leituras recarregadas do CSV. **Não** mexe em última leitura nem em `seq`
`atualizarEstadoAr(id, estado)`	Guarda o estado devolvido por um comando gRPC bem-sucedido
`fonte(id)`, `fontes()`, `totalLeituras()`	Consultas simples

`consultar(tipo, janelaS)`	As 3 agregações sobre as leituras que **chegaram** nos últimos N s (0 = 300 s). `TEMPERATURA_MEDIA`: média dos sensores de temperatura. `RACK_MAIS_QUENTE`: rack de maior média. `PICO_ENERGIA`: maior potência e qual PDU. Sem leituras → `tem_dados = false`
`listar()` / `info(id)`	Visão das fontes para o cliente, com o estado ATIVA/INATIVA calculado **na hora**
`montar(d, agora)` (privado)	**Critério de inatividade:** a fonte está ATIVA se `agora − max(última leitura, descoberta) ≤ 3 × intervalo`, com o intervalo anunciado pela própria fonte

> **Por que 3 intervalos?** Tolera até duas perdas seguidas de UDP sem falso alarme, mas detecta a queda em segundos. O intervalo vem da fonte (e é atualizado a cada leitura), então se o operador mudar o intervalo do ar-condicionado, o critério acompanha. **Por que a hora de chegada e não o `timestamp_ms` da fonte?** Cada máquina tem seu relógio; comparar horas de relógios diferentes dá resultados errados. **Por que as leituras do ar-condicionado não entram nas médias?** Elas medem o ar de saída, não a temperatura de um rack. **Por que a contagem começa na descoberta?** Para uma fonte que nunca enviou nada também ser marcada INATIVA.

5.10 `HistoricoCsv`: persistência

Arquivo `data/leituras.csv`, uma linha por leitura, em ordem de chegada. Colunas: `recebido_em` (ISO, só para leitura humana), `recebido_em_ms`, `fonte_id`, `tipo`, `rack`, `grandeza`, `valor`, `timestamp_ms`, `evento`, `seq`, `intervalo_ms`. Métodos públicos `synchronized`.

Método	O que faz
`gravar(registro)`	Acrescenta uma linha (abre o arquivo e escreve o cabeçalho na primeira vez) e dá `flush`. Se o disco falhar, avisa **uma vez** e segue só em memória: a recepção UDP nunca cai por causa do arquivo
`carregar(desdeMs)`	Lê o arquivo e devolve as leituras com `recebido_em_ms ≥ desdeMs`. Usado na subida do gateway (última hora)
`ultimos(fonteId, limite)`	As últimas *n* leituras do arquivo inteiro, de uma fonte ou de todas. Alimenta o comando `historico`
`lerLinhas`, `interpretar`	Percorrem e convertem as linhas; linha truncada ou editada à mão é **pulada** e contada, sem derrubar nada
`campo`, `dividir`	Escape e leitura de campos no padrão CSV (RFC 4180): aspas só quando há vírgula, aspas ou quebra de linha

> **Por que CSV?** Simples, legível por qualquer editor e planilha, sem dependência nova nem servidor de banco; basta acrescentar linhas ao final (*append*), o que é rápido e seguro. **Por que só a última hora volta para a memória?** É a mesma retenção da memória; assim o consumo não cresce sem limite. **Por que não restaurar a "última leitura" de cada fonte?** Senão uma fonte desligada pareceria ATIVA logo após reiniciar o gateway. **Custo assumido:** o arquivo só cresce e o `historico` lê o arquivo todo (aceitável para esse volume).

5.11 `Cliente`: o terminal do operador

Método	O que faz

`main`	Conecta ao gateway (padrão 127.0.0.1:5009) com **uma conexão que fica aberta**; em laço, lê a linha digitada, monta o pedido, envia e mostra a resposta. `ajuda` e `sair` são locais
`montarPedido(p)`	Traduz o texto digitado em `PedidoCliente`: `listar`; `ligar/desligar <id>`; `setpoint`, `ventilador`, `intervalo`; `consulta media|rack|pico [janela_s]`; `historico [id] [n]`. Erro de digitação mostra a mensagem e continua
`exigir`, `comando`, `numero`	Auxiliares: conferir argumentos, montar `ComandoRequest`, converter número com mensagem clara
`mostrar(resposta)`	Imprime a tabela de fontes, o comando aplicado, o resultado da consulta ou o histórico; `ok = false` imprime `ERRO: …`
`linha(f)`	Formata uma fonte (id, tipo, endereço, rack, estado, hora da última leitura, detalhe do ar-condicionado)

5.12 `Painel`: extra para a demonstração

Servidor HTTP (`com.sun.net.httpserver`, JDK puro) que liga a página `demo-online.html` aos processos Java reais, pois um navegador não consegue subir processos. Escuta só em `127.0.0.1:8099`.

Rota	Função
`GET /`	serve a página, com um token sorteado embutido
`GET /eventos`	fluxo SSE (*Server-Sent Events*): cada linha de saída de um processo vira um evento
`POST /lancar`	sobe `./rodar.sh <Classe> args` num terminal
`POST /entrada`	escreve uma linha no stdin do processo (o `q` / `n` dos sensores, comandos do cliente)
`POST /parar`	encerra o processo do terminal
`POST /quadro-gigante`	teste de robustez: envia ao gateway um quadro TCP declarando 2 GiB

> **Segurança (o professor pode perguntar):** só escuta em loopback; toda chamada exige o token sorteado a cada execução; o cabeçalho `Host` é conferido (contra *DNS rebinding*); só as cinco classes do projeto podem subir, com argumentos filtrados por expressão regular; a página com o token não envia cabeçalhos CORS, então outro site não consegue lê-la. O Painel **não faz parte do sistema distribuído** do enunciado: é só uma conveniência para a demonstração.

6. Fluxos ponta a ponta

6.1 Subida e descoberta

1. Cada fonte sobe e cria a thread de `DescobertaFonte`, que entra no grupo multicast e espera. Ainda **não envia nada**: não sabe o destino.
2. `Gateway.main` recarrega o CSV, sobe as threads UDP/TCP e agenda `descobrir()`.
3. `descobrir()` envia a `DescobertaSolicitacao` (IP do gateway, porta 5008, `nonce`) ao grupo.
4. Cada fonte decodifica o pedido, **guarda o destino das leituras** e responde por unicast com a `DescobertaResposta` (nonce copiado).
5. O gateway, durante 3 s, confere o nonce e chama `cadastro.registrarFonte`. A cada 30 s repete tudo.

Consequência: uma fonte só começa a enviar leituras depois do primeiro pedido de descoberta. Se o gateway subir por último, as fontes passam a enviar logo na primeira rodada.

6.2 Uma leitura (periódica ou evento)

1. A fonte monta a `Leitura` (`seq`, valor, `evento`…) e envia **um datagrama UDP** a gateway:5008.
2. `Gateway.receberLeituras` decodifica e valida; `Cadastro.registrarLeitura` confere o `seq`, guarda e atualiza a última leitura da fonte.
3. `HistoricoCsv.gravar` acrescenta a linha ao CSV. O log mostra `<<< EVENTO` quando `evento = true`.

6.3 Comando ao ar-condicionado

1. Operador digita `setpoint ar-01 20` → `Cliente.montarPedido` cria `PedidoCliente.comando` → `Quadro.enviar` (TCP).
2. `Gateway.atender` recebe → `processar` → `comandar`: acha a fonte no `Cadastro` e abre um canal gRPC até o IP:porta descobertos.
3. A fonte executa `Servico.configurar`: `validar` → `aplicar` → responde com o novo `EstadoAr` (ou `INVALID_ARGUMENT`).
4. O gateway guarda o estado (`atualizarEstadoAr`) e responde ao cliente `ok = true` com a `FonteInfo` atualizada, ou `ok = false` com o motivo.

6.4 Consulta agregada e histórico

1. `consulta media 10` → `ConsultaRequest(TEMPERATURA_MEDIA, 10)` → `Cadastro.consultar` percorre o histórico em memória (só leituras que chegaram nos últimos 10 s) → `ConsultaResponse`.
2. `historico temp-rack-A 3` → `HistoricoRequest` → `Gateway.historico` → `HistoricoCsv.ultimos` (lê o arquivo) → `HistoricoResponse`.
3. Diferença: a **consulta** usa a memória (última hora); o **histórico** usa o arquivo (inclui execuções anteriores).

6.5 Reinício do gateway

1. O gateway é encerrado: o `Cadastro` (memória) se perde; o CSV permanece.
2. Ao subir de novo, `main` chama `csv.carregar(agora − 1 h)` e `cadastro.restaurar(…)`: as consultas por janela voltam a funcionar.

3. As fontes são redescobertas na primeira rodada (até 30 s no pior caso, com o intervalo padrão); as fontes que continuaram enviando voltam ao `listar` assim que cadastradas.

6.6 Falha de uma fonte

1. O processo da fonte é morto: param as leituras UDP (e o gRPC, se for o ar-condicionado).
2. Comando a essa fonte → `UNAVAILABLE` (após no máximo 3 s) → o cliente vê "fonte … não respondeu"; o gateway continua de pé.
3. Depois de mais de **3 intervalos** sem leitura (cerca de 6 s com o intervalo de 2 s), `listar` mostra a fonte como `INATIVA`. Se ela voltar, vira `ATIVA` na próxima leitura.

7. Concorrência: quem roda em qual thread

Processo	Threads	Estado compartilhado e proteção
Gateway	`udp-leituras`; `tcp-clientes` + uma thread por cliente; agendador da descoberta	`Cadastro` (`synchronized`) e `HistoricoCsv` (`synchronized`, outro cadeado)
ArCondicionado	principal (leituras); `descoberta`; pool do gRPC	estado do ar (`synchronized`); `gateway` é `volatile`
SensorTemperatura / Pdu	principal; `descoberta`; `teclado`	`base` e `gateway` são `volatile` (uma thread escreve, outra lê)
Cliente	uma só	nenhum

- **Sem deadlock:** o gateway pega o cadeado do `Cadastro` e depois, em outro passo, o do `HistoricoCsv`; nunca um dentro do outro.
- **Por que uma thread por cliente TCP?** Simples e suficiente para um operador (ou poucos). Com centenas de clientes valeria um pool ou E/S assíncrona. **Por que não usar uma thread só para tudo?** `accept` e `receive` bloqueiam: uma thread parada esperando um cliente impediria de receber leituras.
- **`volatile`** garante que a thread que lê enxergue o valor que a outra escreveu; **`synchronized`** garante exclusão mútua em operações com mais de um passo.

8. Decisões e justificativas (perguntas prováveis)

O professor deve perguntar o **motivo** de cada decisão. Respostas curtas, prontas para falar.

1. Por que Java?

Uma linguagem só para os cinco processos, com sockets, threads e multicast na biblioteca padrão, e suporte maduro a Protobuf e gRPC. O trabalho anterior da disciplina já era em Java (sockets e Protobuf), então reaproveitei o conhecimento. Tipagem estática ajuda aqui: o código gerado do `.proto` é fortemente tipado e o compilador acusa campo errado. O enunciado trata várias linguagens como pontuação extra, não como requisito, então priorizei o escopo mínimo bem feito.

2. Por que Protocol Buffers?

É exigido pelo enunciado, e também é uma boa escolha: formato binário compacto (importante em UDP, onde cada datagrama é uma mensagem), contrato único e explícito (`.proto`), geração automática de código para as duas pontas e evolução compatível (campos novos com números novos não quebram quem ainda não os conhece; foi o que aconteceu ao acrescentar o histórico).

3. Por que um único `.proto` para tudo?

Garante que todos os processos falem exatamente a mesma língua e que uma mudança no contrato apareça em tempo de compilação em todos. Os tipos comuns (`EstadoAr`, enums) são reutilizados em vários canais.

4. Por que multicast na descoberta?

O gateway não conhece os IPs das fontes. Com multicast ele faz uma pergunta a um grupo e quem estiver escutando responde, sem lista de endereços e sem configuração. Fontes que entram depois são achadas na rodada seguinte.

5. Por que a resposta da descoberta é unicast, e por que existe o *nonce*?

Só o gateway precisa da resposta; multicast na volta geraria tráfego inútil para todas as fontes. O *nonce* (número sorteado por rodada, devolvido pela fonte) impede que o gateway confunda uma resposta atrasada de uma rodada anterior com a atual.

6. Por que UDP para as leituras?

São frequentes, pequenas e a perda de uma leitura é tolerável (a próxima chega em 2 s). UDP não exige conexão por fonte nem consome recursos do gateway por dispositivo, e combina com dispositivos simples. O preço é que a rede pode perder, duplicar ou reordenar datagramas: por isso a leitura leva um `seq` e o gateway o confere e registra a anomalia.

7. Por que o evento imediato existe?

Esperar até 2 s para avisar que o rack passou de 27 °C é tempo demais num data center. O sensor envia uma leitura extra, com `evento = true`, no instante em que cruza o limiar (e de novo quando volta ao normal).

8. Por que gRPC para o comando e não UDP ou TCP "à mão"?

Comando exige resposta confiável e verificável. O gRPC dá contrato tipado (definido no `.proto`), códigos de erro padronizados (`INVALID_ARGUMENT`, `UNAVAILABLE`), prazo por chamada e geração de cliente e servidor. Também é um requisito do enunciado para a fonte controlável.

9. Por que TCP com 4 bytes de tamanho entre cliente e gateway?

TCP é um fluxo de bytes, sem fronteira de mensagem, e o Protobuf não indica onde termina. O prefixo de 4 bytes diz quantos bytes vêm a seguir. O limite de 1 MiB protege o gateway de um tamanho absurdo. (Alternativa válida: `writeDelimitedTo` do Protobuf, que usa um tamanho em *varint*; escolhi 4 bytes fixos porque é independente de biblioteca e fácil de explicar e de mostrar em hexadecimal.)

10. Por que `optional` e `oneof` no contrato?

`optional` distingue "campo ausente" de "zero" e permite atualização parcial no comando. `oneof` permite que um único tipo de mensagem no TCP carregue pedidos diferentes, e o compilador força a tratar cada caso.

11. Por que o comando carrega valores absolutos?

Repetir um comando por causa de um timeout não tem efeito colateral: "setpoint = 20" repetido continua 20. É a propriedade de idempotência, importante numa rede em que não se sabe se o comando chegou.

12. Por que a fonte valida o comando?

Quem conhece os limites do equipamento é a própria fonte. O gateway repassa; se a fonte recusar, devolve `INVALID_ARGUMENT` e o gateway entrega o motivo ao operador, sem duplicar regras nem divergir.

13. Como o gateway sabe que uma fonte caiu? Por que 3 intervalos?

Fonte sem nenhuma leitura há mais de 3 intervalos **anunciados por ela** é marcada `INATIVA`. Três intervalos tolera duas perdas seguidas de UDP sem falso alarme e ainda detecta a queda em poucos segundos. Limitação: o critério por tempo não distingue fonte caída de rede caída.

14. Por que o ar-condicionado continua enviando leituras quando é desligado?

Para distinguir "desligado por comando" de "caiu". Se parasse de enviar, as duas situações seriam idênticas para o gateway.

15. Por que o gateway usa a hora de chegada, e não o `timestamp_ms` da fonte?

Cada máquina tem um relógio, possivelmente desacertado. Para decidir "isso aconteceu nos últimos 10 s" e "essa fonte está viva", o gateway precisa de uma única referência de tempo: o seu próprio relógio.

16. Por que 27 °C?

É o topo da faixa de temperatura de entrada recomendada pela ASHRAE (TC 9.9) para equipamentos de TI. Acima disso o risco para o equipamento aumenta.

17. Por que `synchronized` no `Cadastro` inteiro, e não um cadeado por estrutura?

Simplicidade e correção: as operações são curtas e o volume é baixo (algumas leituras por segundo), então a contenção é desprezível. Cadeados finos aumentariam o risco de deadlock e de inconsistência entre as estruturas.

18. Por que guardar as leituras em CSV?

Para o histórico sobreviver a reinício do gateway, sem trazer um banco de dados. CSV é legível por qualquer ferramenta e escrever no final do arquivo é barato. Ao reiniciar, só a última hora volta para a memória (igual à retenção em memória) e a "última leitura" das fontes **não** é

restaurada, para que uma fonte desligada não pareça ativa. O arquivo cresce indefinidamente; num uso longo seria preciso rotacioná-lo ou usar um banco.

19. Por que as fontes são processos separados?

O enunciado pede partes independentes que se comunicam por rede. Processos separados falham, reiniciam e são desligados de forma independente, o que permite demonstrar a detecção de falha e a descoberta de fontes que entram depois.

20. Por que as consultas ignoram o ar-condicionado?

A temperatura que ele reporta é a do ar de saída, não a de um rack. A "temperatura média" e o "rack mais quente" são sobre racks, então só os sensores de rack entram.

21. E o que acontece se a janela da consulta não tiver leituras?

A resposta traz `tem_dados = false` e o cliente escreve "sem dados nos últimos N s". Não se inventa um valor (como 0,0).

9. Os dispositivos (fontes de dados)

O professor disse que, se o trabalho tem dispositivos IoT, vai fazer perguntas sobre eles e pediu um vídeo. As fontes deste trabalho são **dispositivos simulados por software**: cada um é um processo Java independente que se comporta como o equipamento real, e o protocolo que falam é o que um dispositivo físico implementaria. Diga isso com franqueza.

Dispositivo	Representa	Controlável?	O que envia	Como mexer na demonstração
`ar-01`	Ar-condicionado de precisão (CRAC)	Sim (gRPC)	Temperatura do ar a cada 2 s + estado (`EstadoAr`)	`ligar` , `desligar` , `setpoint` , `ventilador` , `intervalo`
`temp-rack-A/B`	Sensor de temperatura de rack	Não	°C a cada 2 s; **evento imediato** ao cruzar 27 °C	digitar `q` (esquenta) ou `n` (normal) no terminal do sensor
`pdu-rack-A/B`	PDU (distribuição de energia do rack)	Não	Potência em W a cada 2 s	digitar `q` (pico de 6500 W) ou `n` no terminal da PDU

Perguntas prováveis sobre os dispositivos

- **Como o dispositivo sabe para onde enviar?** Aprende com o pedido de descoberta (`gateway_ip` e `gateway_porta_udp`). Antes do primeiro pedido, não envia nada.
- **Como o gateway identifica o dispositivo?** Pelo `fonte_id` único, escolhido na linha de comando; a descoberta informa tipo, IP, porta e estado inicial.
- **Qual o tamanho de uma leitura?** Algumas dezenas de bytes por datagrama (dá para ver os bytes reais ligando "Mostrar bytes" na simulação da demonstração).
- **O que acontece se o dispositivo reinicia?** O `seq` volta a 0 e o gateway trata isso como reinício, sem alarme. Na próxima rodada de descoberta ele é recadastrado.
- **E se o dispositivo ficar sem energia/rede?** Param as leituras; após 3 intervalos fica `INATIVA` ; se for o ar-condicionado, comandos retornam `UNAVAILABLE` .
- **Por que simulados?** O foco da disciplina é a comunicação distribuída; simular o equipamento permite reproduzir falhas e picos na hora da demonstração e rodar tudo num notebook.

- **Os dados são reais?** Não: valores gerados por um modelo simples (temperatura que tende ao setpoint com ruído; consumo com ruído em torno de uma base).

> **Vídeo.** O vídeo de até 7 minutos (processos iniciando, cliente em uso e uma falha simulada) é parte da entrega e já tem roteiro. Tenha o link aberto na arguição.

10. Limitações que vale admitir

Admitir o limite com clareza costuma soar melhor do que ser surpreendido. As principais, todas conhecidas:

- **Gateway é ponto único de falha.** Se cai, o cadastro de fontes se perde (volta na próxima rodada de descoberta, até 30 s); o histórico persiste no CSV.
- **Sem TLS nem autenticação:** qualquer um na rede pode falar com o gateway, inclusive enviar leituras falsas por UDP. Em produção usaríamos TLS no gRPC e no TCP.
- **Detector por tempo** não distingue fonte caída de rede caída.
- `ObterEstado` **existe no contrato e na fonte, mas o gateway não o chama:** o estado chega pela descoberta, pelo `estado_ar` anexado às leituras e pela resposta do `Configurar`. Foi mantido por completar o serviço.
- **Leituras de fontes não descobertas** também são aceitas e entram no histórico e nas consultas (a validação é só de campos básicos).
- **Um canal gRPC por comando** e **uma thread por cliente TCP**: simples, mas não escala para muitos comandos ou clientes.
- **CSV cresce sem limite** e `historico` lê o arquivo inteiro.
- **Multicast depende da rede:** algumas redes (Wi-Fi com isolamento de clientes, redes corporativas) bloqueiam. Veja o roteiro da seção 12.

11. Mapa: pergunta → onde está no código

Se perguntarem…	Mostre
Onde está o contrato?	`src/main/proto/datacenter.proto`
Onde o gateway envia o pedido de descoberta? E confere o nonce?	`Gateway.descobrir()`
Como a fonte responde à descoberta?	`DescobertaFonte.run()`
Onde chega uma leitura UDP?	`Gateway.receberLeituras()`
Como detecta perda/duplicação de UDP?	`Cadastro.registrarLeitura()` (confere o `seq`)
Onde está o critério de inatividade?	`Cadastro.montar()` e `Config.TOLERANCIA_INATIVIDADE`
Onde o sensor dispara o evento do limiar?	`SensorTemperatura.enviarLeituras()` (variável `evento`)
Onde o gateway chama o gRPC?	`Gateway.comandar()`
Onde o ar-condicionado valida e aplica?	`ArCondicionado.validar()`, `aplicar()`, `Servico.configurar()`
Como o TCP separa as mensagens?	`Quadro.enviar()` / `Quadro.receber()`

Onde as 3 consultas são calculadas?	`Cadastro.consultar()`
Onde o CSV é gravado e recarregado?	`HistoricoCsv.gravar()`, `carregar()`; uso em `Gateway.main()` e `receberLeituras()`
Como o cliente interpreta os comandos?	`Cliente.montarPedido()`
Como o multicast escolhe a placa de rede?	`Rede.escolherInterface()`

12. Antes da arguição: deixar tudo rodando

O professor pediu para **deixar tudo configurado antes de começar**, para ver rodando. Faça o ensaio completo uma vez, no mesmo notebook e na mesma rede da arguição.

Checklist (do dia anterior até a hora)

1. **Atualizar o código:** `git pull` e `mvn -q compile` (não deve imprimir nada; precisa de internet na primeira vez).
2. **Limpar processos antigos:** `pkill -f 'java.*dc[.]'`.
3. **Escolher a forma de subir:**
   - **Painel (mais prático):** `./rodar.sh Painel`, abrir `http://localhost:8099/` no Chrome, conferir "Java real conectado" e clicar **Subir tudo**.
   - **7 terminais (mais "cru"):** na ordem do `README`: ar-condicionado, 2 sensores, 2 PDUs, **gateway**, cliente.
4. **Conferir a rede (o ponto mais frágil).** O multicast precisa de uma rede ativa que o permita. Se o notebook tiver mais de uma interface, descubra o nome com `ip -br a` e use o **mesmo** `JAVA_OPTS="-Ddc.interface=<nome>"` em todos os processos. Teste **sem internet** e na rede da sala (Wi-Fi institucional costuma bloquear multicast). Alternativa: hotspot do celular ou cabo.
5. **Conferir o resultado:** no cliente, `listar` deve mostrar **5 fontes ATIVAS**. Sinal de que deu certo: o log do gateway com `[descoberta] nova fonte` cinco vezes.
6. **Decidir sobre o histórico:** se quiser começar do zero, apague `data/leituras.csv` antes de subir. Se preferir mostrar dados de ensaios, mantenha.
7. **Deixar prontos para mostrar:** `listar` já executado; os terminais do sensor A (para digitar `q`) e da PDU B; `historico 5` pré-digitado se possível.
8. **Deixar abertos:** `slides.pdf`, este guia, o repositório e os arquivos-chave no editor (`datacenter.proto`, `Gateway.java`, `Cadastro.java`).
9. **Deixar o vídeo** aberto (link testado em janela anônima).
10. **Notebook:** carregador na tomada, notificações desligadas, zoom do terminal legível, tela espelhada ou compartilhada testada.

Plano B se o multicast falhar na sala

- Usar a **simulação** da página `demo-online.html` (botão "Simulação"): não depende de rede e mostra os mesmos comportamentos, inclusive os bytes reais do Protobuf. O endereço público fica no formulário de entrega (GitHub Pages).
- Mostrar o **vídeo** gravado com tudo funcionando e explicar o que falhou (rede bloqueando multicast); é um problema de ambiente conhecido, não do código.

Roteiro de demonstração (2 a 3 minutos)

1. `listar` → 5 fontes ATIVAS (descoberta funcionou).
2. No terminal do sensor A, `q` → aparece `<<< EVENTO` no gateway na hora (UDP + evento imediato). `n` → novo evento.
3. `setpoint ar-01 99` → recusado (`INVALID_ARGUMENT`); `setpoint ar-01 20` → aceito (gRPC).
4. Na PDU B, `q`; depois `consulta media 10`, `consulta rack 10`, `consulta pico 10` (TCP e agregações).
5. `historico temp-rack-A 5`; reiniciar o gateway; repetir (persistência).

6. Matar o ar-condicionado; `setpoint ar-01 21` → `UNAVAILABLE`; após alguns segundos `listar` → `ar-01 INATIVA` (falha).

Dividir a fala

Veja o arquivo `docs/DIVISAO_SLIDES.md`: quem apresenta cada slide e o que dizer. Combinem também **quem opera os terminais** enquanto o outro explica.